

Calepin

Computational Notebooks and Static Websites in Typst

Computational notebooks

Write in Typst, execute code, and see results inline. Perfect for data analysis, reports, and publications with reproducible outputs, in the spirit of literate programming.

- Typst-native authoring
- Executable code chunks
- Inline results and plots
- Python, R, Julia, Bash, and all Jupyter kernels
- Export to HTML and PDF

Calepin: Computational notebooks in Typst

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Quam quisque in leo. Ut enim aenean ante sollicitudin. Ut enim aenean ante sollicitudin. Ut enim aenean ante sollicitudin.

```
from plotnine import ggplot, aes, geom_point, labs
from plotnine.data import mtcars
{
  ggplot(mtcars, aes("mpg", "hp"))
  + geom_point(color = "orange")
  + labs(x="Miles per gallon", y="Horsepower")
}.show()
```

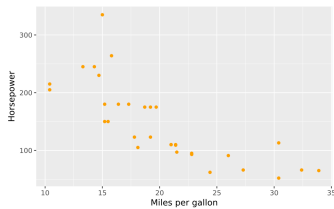
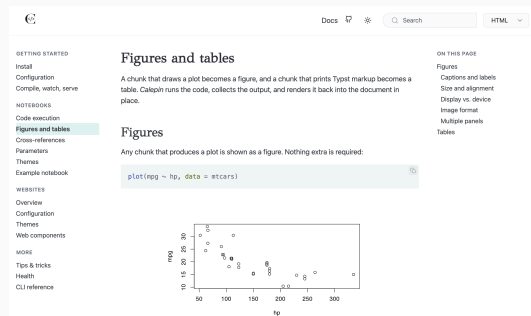


Figure 1: Plotnine scatterplot

Static website generator

Build multi-page websites from Typst with ease. Great for docs, portfolios, and project sites.

- Themes, templates and layouts
- Reusable web components
- Fast incremental builds
- Live preview
- Multi-lingual support
- Blog listings and feeds
- Search
- ...and much more!

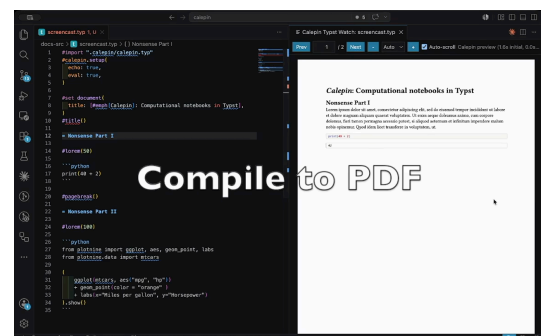


Pure Typst

Write notebooks in pure Typst, a simple, consistent, powerful, and elegant typesetting system. There is no special file format; notebooks and websites are just standard .typ documents. You do not need to “declare” your markup as Typst using special “fences.” *Calepin* does not push your text through a lossy Pandoc translation layer, and you do not need to learn yet another *ad hoc* markdown variant.

Editor integration

Write, execute, preview, and publish Calepin documents without leaving your editor. Install the Calepin extension from the VS Code Marketplace, or from Open VSX for Cursor, Positron, and other VSX-compatible editors.



A simple computational notebook

```
#import "@preview/calepin:0.0.1" as calepin
```

```
#calepin.setup(  
  echo: true,  
  results: "verbatim",  
)
```

```
#let py = calepin.inline.with("python")
```

```
```python  
x = 41
print(x + 1)
```
```

Variables are persistent across chunks:

```
```python  
print(x + 2)
```
```

The inline answer is `#py[`print(40 + 2)`]`.